Assignment# 05

Assignment Instructions

Students are required to develop Python programs for the following real-world scenarios.

1. The first three questions are compulsory. Bonus questions are optional and will earn additional marks.
2. Use functions wherever appropriate to organize your programs.
3. Apply exception handling to manage invalid inputs and file-related errors.
4. Use suitable built-in Python modules and create custom modules/packages where required.
5. Apply previously studied concepts, including lists, tuples, dictionaries, sets, *args, **kwargs, lambda expressions, map(), filter(), reduce(), and zip() wherever applicable.
6. Include meaningful comments in your source code.
7. Submit the Python source files and any required input/output files in an organized folder.
8. Submit the zip file with name RollNo_Assignment05.zip having all the files.
9. Before submitting, verify that the ZIP file opens correctly and contains all required files.
   Warning: Corrupt, incomplete, or inaccessible ZIP files will receive zero marks.

Important:
No deadline extensions will be granted. Submit well before the deadline.
Internet connectivity issues or other last-minute technical problems will not be accepted as excuses for late submission.

Question 1: University Student Result and Exception Management System

A university wants to develop a Python application to manage student examination results. The application should accept student information and marks in multiple subjects, calculate results, classify student performance, and save the reports in a file.

Requirements

Write a Python program that performs the following tasks:

A. Accept student names, roll numbers, and marks in multiple subjects. Store the information in a dictionary, using roll numbers as keys.

B. Create a function that accepts a student's name and a variable number of subject marks using *args. Calculate total marks and average marks.

C. Use map() with a lambda expression to calculate subject-wise percentages and filter() to identify subjects with marks greater than or equal to 80.

D. Classify students into the following categories using conditional statements:

- Excellent: Average ≥ 80
- Good: Average ≥ 60 and less than 80
- Satisfactory: Average ≥ 50 and less than 60
- Needs Improvement: Average below 50.

E. Implement exception handling to manage the following situations:

- Marks outside the range 0–100.
- Non-numeric marks.
- Missing student information.
- Division by zero when no subject marks are provided.

F. Create a user-defined exception named InvalidMarksError and raise it whenever a student enters marks outside the permitted range.

G. Save the student result report in a text file. Use appropriate file handling techniques and ensure that the file is properly closed, even if an exception occurs.

H. Display the final result report containing student information, subject-wise marks, total, average, percentages, high-scoring subjects, and performance category.

Question 2: Smart Business Utility and Reporting System

A small business wants to develop a Python application that provides useful calculations, generates random transaction IDs, displays current date and time, and creates monthly business reports.

The application should use Python built-in modules and user-defined modules to organize its functionality.

Requirements

Write a Python program that performs the following tasks:

A. Create a Python module named business_utils.py containing functions for:

- Calculating the total and average sales.
- Calculating the square root of a sales value using the math module.
- Generating a random transaction ID using the random module.
- Displaying the current date and time using the datetime module.

B. Create another module named report_utils.py that contains a function to generate a monthly calendar using the calendar module.

C. Create a main Python program that imports and uses the functions from both modules.

D. Accept daily sales figures from the user and store them in a list. Use map() with a lambda expression to calculate a 5% sales tax for each transaction.

E. Use filter() to identify sales transactions greater than Rs. 10,000 and reduce() to calculate the total sales amount.

F. Use zip() to combine transaction IDs with their corresponding sales amounts and store the results in a dictionary.

G. Use the time module to display the time taken to generate the report.

H. Save the generated report in a text file. Use appropriate exception handling to manage errors during module imports and file operations.

I. Demonstrate how Python locates imported modules and explain how a module can be reloaded using importlib.reload().

Question 3: Library Inventory and File Management System

A university library wants to develop a Python application to manage its book inventory. The application should maintain book information, search for books, update inventory records, and save information in files.

Write a Python program that performs the following tasks:

A. Store book records in a dictionary. Each book should contain its ID, title, author, category, and available quantity.

B. Create a user-defined Python module named book_operations.py containing functions to:

- Add a new book.
- Search for a book by its ID or title.
- Update the available quantity.
- Display all available books.

C. Create a package named library containing the following modules:

- book_operations.py
- file_operations.py

Import and use the functions from these modules in the main application.

D. Use *args to accept multiple book titles for searching and **kwargs to accept book information when adding a new book.

E. Use lambda expressions with filter() to identify books belonging to a particular category or books having an available quantity greater than zero.

F. Save book records in a text file. Implement the following file operations:

- Write new book records.
- Read records using read(), readline(), or readlines().
- Append newly added books without deleting existing records.
- Use seek() to change the file reading position.

G. Use the os module to:

- Create a directory named LibraryRecords.
- Check whether a file exists.
- Display file attributes using os.stat().
- Rename or delete a file.

H. Implement exception handling for missing files, invalid book IDs, and file access errors.

I. Display the complete library inventory and save the updated inventory report.

Bonus Questions

Question 4: Digital Image Processing and Management Application

A university media department wants to develop a Python application to manage and process images. The application should allow users to open images, inspect their properties, perform basic image transformations, and save the processed images.

Requirements

Write a Python program using the Pillow (PIL) library to perform the following tasks:

A. Accept multiple image filenames from the user and store them in a list.

B. Open an image using the Image.open() function and display its basic attributes, including:

- Image filename.
- Image format.
- Image dimensions.
- Image mode.

C. Perform the following image processing operations:

- Resize an image.
- Crop a selected region.
- Rotate an image.
- Convert an image into grayscale.

D. Use map() with a lambda expression to apply a selected image transformation to multiple image files.

E. Use filter() to identify images having dimensions greater than 500 × 500 pixels.

F. Use the os module to create a directory named ProcessedImages and save all processed images in that directory.

G. Use the os module to check image file existence and display file information using os.stat().

H. Implement exception handling to manage:

- Image file not found.
- Unsupported image format.
- Invalid crop coordinates.
- Errors while saving processed images.

I. Display the names of original and processed images.

Note: Install Pillow before running the program using pip install Pillow.

Question 5: Employee Attendance and Payroll Report Generator

A software company wants to develop a Python application to manage employee attendance and calculate monthly working hours. The application should process attendance records, calculate employee working hours, identify employees with insufficient attendance, and generate a monthly report.

Requirements

Write a Python program that performs the following tasks:

A. Store employee information in a dictionary containing employee ID, name, department, and daily attendance records.

B. Use the datetime and calendar modules to determine the current month, number of days in the month, and dates of working days.

C. Create a function that accepts an employee ID and a variable number of daily working hours using *args. Calculate the employee's total and average working hours.

D. Use map() with a lambda expression to calculate the daily salary of employees, assuming an hourly rate of Rs. 500.

E. Use filter() to identify employees whose average daily working hours are less than 6 hours.

F. Use zip() to combine employee names with their corresponding total working hours. Store the results in a dictionary.

G. Use reduce() to calculate the total salary expenditure of the company.

H. Save employee attendance and payroll information in a text file. Use file handling techniques to append new attendance records and read existing records.

I. Use the threading module to create two separate threads:

- Thread 1: Calculate employee working hours.
- Thread 2: Generate the employee payroll report.

J. Implement exception handling for invalid attendance records, missing employee IDs, and file handling errors. Display a complete monthly attendance and payroll report.